Control Engineering Practice 156 (2025) 106213



Contents lists available at ScienceDirect

Control Engineering Practice

journal homepage: www.elsevier.com/locate/conengprac





Hierarchical Deep Reinforcement Learning for cubesat guidance and control

Abdulla Tammam *, Nabil Aouf

City, University of London, Northampton Square, London, EC1V 0HB, UK

ARTICLE INFO

Keywords:
Reinforcement learning
Deep learning
Spacecraft guidance and control
Hardware-in-the-Loop
Intelligent control

ABSTRACT

Advancements in Reinforcement Learning (RL) algorithms and technologies have opened up new possibilities for their use in autonomous spacecraft control. This work presents a novel Hierarchical Deep Reinforcement Learning (HDRL) agent which can autonomously achieve satellite rendezvous while maintaining attitude control. The HDRL agents presented are built on a Hierarchical Actor–Critic (HAC) framework and are compared against combined and distributed TD3 RL agents. The controller has demonstrated the ability to achieve satellite rendezvous while performing large-angle slew manoeuvres with pointing accuracies of less than five degrees and resisting environmental perturbations. To assess the controller's feasibility a six-Degree-of-Freedom (6-DoF) spacecraft dynamics testing platform was designed and constructed. The platform is made up of a reaction wheel actuated mock CubeSat, a frictionless space environment setup for attitude testing and a robotic arm based rendezvous mission simulator.

1. Introduction

In recent years there has been a rapidly growing demand for space-based services such as Earth observation, on-orbit servicing, and active debris removal, necessitating the need for advanced spacecraft control systems. These systems should be capable of effectively addressing complex challenges associated with spacecraft guidance and attitude control, particularly in dynamic and uncertain environments. These systems are especially important in environments where spacecraft are more susceptible to disturbances that are not modelled completely such as air drag and space debris. These disturbances, whether gradual and sustained (Gravity gradient torque, aerodynamic torque, etc.) or violent and sudden (space debris, orbital waste disposal, etc.), could cause unwanted changes in the orientation, angular velocity or orbit of the spacecraft. No matter the cause, it is vital to design a control system that can correct these changes while complying with performance requirements and actuator limitations.

Traditional control systems, which generally rely on ground-based mission control, are limited by communication delays and inflexibility of the desired task. This highlights the need for a new generation of control systems that can adapt in real-time to changing conditions. Autonomous control systems leverage machine learning algorithms and artificial intelligence to continuously update their understanding of the environment, enabling them to respond to unforeseen challenges more effectively. By reducing reliance on ground-based mission control, these systems enable spacecraft to undertake a broader range of tasks and react more effectively to contingencies, a crucial factor for missions where communication with Earth is challenging either due to large distances or time constraints.

Active debris removal missions such as the European Space Agency's e.Deorbit mission exemplify the need for these new types of controllers (Jaekel et al., 2018). This mission type requires accurate tracking and capture of tumbling space debris while minimising any risk of collision and fragmentation. In this case, autonomous rendezvous and attitude control systems would allow the spacecraft to adapt to the dynamic environment surrounding the debris and execute intricate capture manoeuvres that are not only responsive to the target object's motion but also to any possible hazards that could jeopardise the mission. The utilisation of autonomous control systems can not only improve mission success rates but also reduce the reliance on ground-based mission control. This independence paves the way for more complex and ambitious space exploration endeavours, empowering spacecraft to undertake a wider range of tasks and respond effectively to unforeseen challenges.

In this paper, a novel hierarchical deep reinforcement learning (HDRL) controller is presented, built, and applied to the problem of providing simultaneous guidance and attitude control for short-range rendezvous operations. The HDRL controller discussed in this paper is constructed using a hierarchical actor–critic (HAC) framework combined with the deep deterministic policy gradient (DDPG) algorithm. This innovative approach enables the efficient learning of complex behaviours by employing multiple policies operating at different time resolutions. The HDRL controller is designed to overcome the limitations of conventional control systems by harnessing the power of

* Corresponding author.
E-mail address: abdulla.tammam@city.ac.uk (A. Tammam).

https://doi.org/10.1016/j.conengprac.2024.106213
Received 30 July 2024; Received in revised form 12 October 2024; Accepted 9 December 2024
Available online 19 December 2024
0967-0661/© 2024 Published by Elsevier Ltd.

Table 1
Table of Acronyms.

CW	Clohessy–Wiltshire
DDPG	Deep Deterministic Policy Gradient
DoF	Degrees of Freedom
DRL	Deep Reinforcement Learning
EPOS	European Proximity Operations Simulator
HAC	Hierarchical Actor-Critic
HDRL	Hierarchical Deep Reinforcement Learning
HER	Hindsight Experience Replay
HIL	Hardware-in-the-loop
IMU	Inertial Measurement Unit
LEO	Low Earth Orbit
LTI	Linear Time-Invariant
MDP	Markov Decision Process
MSBE	Mean Squared Bellman Error
PD	Proportional–Derivative
PWM	Pulse Width Modulation
RvD	Rendezvous and Docking
RL	Reinforcement Learning
TD3	Twin Delayed Deep Deterministic Policy Gradient
UVFA	Universal Value Function Approximator
UAV	Unmanned Aerial Vehicle

advanced reinforcement learning techniques, enabling the autonomous control of spacecraft in uncertain and dynamic environments.

To validate the effectiveness and advantages of the HDRL controller its performance is compared with a classical controller, a decentralised and a centralised Twin Delayed DDPG (TD3) based controllers. This comparative analysis demonstrates the advantages offered by the HDRL controller, including improved efficiency and enhanced overall performance in solving complex spacecraft control challenges. Additionally, a hardware testbed was designed and constructed in order to allow for hardware-in-the-loop (HIL) testing of the proposed controller. This represents the first presentation of a deep reinforcement learning (DRL) controller being tested with HIL for satellite guidance and attitude control.

The paper is divided into several key sections. In Section 2 the existing body of research is explored and critically examined, identifying gaps in the research and highlighting key contributions. Next, Section 3 presents an overview of the fundamental concepts, terminologies, and algorithms that will be utilised in this paper. Section 4 presents the guidance and attitude problems that the controllers are designed to solve. In Section 5, the methodology, the agents that serve as the basis for the controllers are introduced, followed by a description of their design and structure. Following this, in Section 6, the agents are moved to the simulation environment where they are tested and compared, subsequently in Section 7 the hardware testbed is designed, constructed and used to validate the chosen controller. Lastly, in Section 8, conclusions are drawn and future work is proposed.

Ultimately, the development and implementation of advanced control systems, such as the HDRL controller presented in this paper, hold great potential for improving the autonomy, safety, and efficiency of future space missions. As we continue to push the boundaries of space exploration and utilisation, the need for intelligent and adaptive control systems will only grow. The HDRL approach offers a promising avenue for addressing these challenges and enabling more innovative space missions.

2. Literature review

2.1. The control problems

The attitude control problem, also known as the Euler rigid body problem, involves determining and implementing the necessary adjustments to a spacecraft's orientation in order to achieve and maintain a desired orientation. Adding to the challenge, the dynamics of the spacecraft and its environment are often incompletely modelled and time

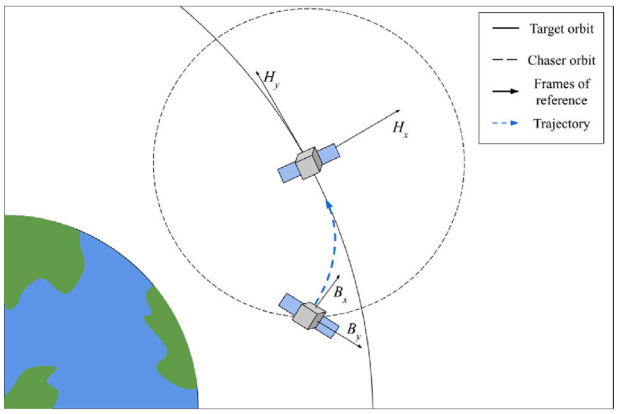


Fig. 1. An illustration of the Hill frame commonly used for the Clohessy–Wiltshire equations.

varying which can make precise control of the spacecraft's behaviour more difficult (Tipaldi, Iervolino, & Massenio, 2022). The current state of the art uses feedback control schemes such as adaptive control (Luzi, Peaucelle, Biannic, Pittet, & Mignot, 2014; Navabi & Radaei, 2013), fuzzy control (Cheng, Shu, & Cheng, 2009; Cihang & Jang, 1994), sliding mode control (Gui & Vukovich, 2017; Tiwari, Janardhanan, & un Nabi, 2015), and model predictive control (Gupta, Kalabić, Di Cairano, Bloch, & Kolmanovsky, 2015; Hegrenæs, Gravdahl, & Tøndel, 2005). The problem with these methods is that, without adjusting their parameters, high performance can only be achieved with the spacecraft they were constructed on, known as the target spacecraft (Zheng, Wu, & Li, 2021). This in turn means that any change in the properties of the spacecraft, such as its mass or inertia matrix, could render the controller unsuitable. Modern missions such as active debris removal, target capture and on orbit servicing would cause large shifts in a spacecraft's properties (Huang & Duan, 2020) highlighting the need for more advanced controllers for these types of missions

The rendezvous problem involves manoeuvring a spacecraft to approach, match, and maintain a desired relative position and velocity with respect to a target spacecraft. One of the most common classical methods used for rendezvous control is Clohessy–Wiltshire (CW) targeting; this method was used by spacecraft, such as Soyuz, Progress, ATV, HTV, and Shenzhou (Luo, Zhang, & Tang, 2014). CW targeting uses the CW equations to determine two optimal impulsive manoeuvres required to reach the desired state. (see Fig. 1). Alternatively, the glideslope targeting method is a constant direction guidance law that expands the CW targeting method to include constant relative accelerations (Pearson, 1989). Modern methods include linear quadratic control (Mazal, Pérez, Bevilacqua, & Curti, 2016), fuzzy control (Karr & Freeman, 1997; Ortega, 1995) and model predictive control (Fear & Lightsey, 2021; Li, Yuan, Zhang, & Gao, 2017).

2.2. Reinforcement learning

Reinforcement learning (RL) is a powerful machine learning technique that has shown promise in the field of autonomous robots, particularly for drones and Unmanned Aerial Vehicles (UAVs) (AlMahamid & Grolinger, 2022; Azar et al., 2021; Pham, La, Feil-Seifer, & Nguyen, 2018). By employing RL techniques, drones have been able to achieve complex tasks such as; autonomously navigate complex environments with limited depth information (He, Aouf, Whidborne, & Song, 2020), and avoiding obstacles in limited or unknown environments (Pham et al., 2018; Singla, Padakandla, & Bhatnagar, 2021). Advancements in this sector have enabled drones to operate with higher levels of autonomy and efficiency, leading to a broader range of applications, such as disaster response, agricultural surveying, and surveillance.

The success of RL in the realm of drones and autonomous robots, along with its adaptability to changing conditions and uncertainties, have led to similar techniques being applied to spacecraft control (Izzo, Martens, & Pan, 2018; Shirobokov, Trofimov, & Ovchinnikov, 2021; Song, Rondao, & Aouf, 2022; Tipaldi & Glielmo, 2018). Traditional control methods generally require a priori knowledge of the system dynamics and a pre-designed controller, which may not always be feasible for complex spacecraft systems. On the other hand, RL-based controllers are able to adapt to changes in the spacecraft system and environmental conditions, and tackle systems with complex nonlinear dynamics and stochastic perturbations (Shirobokov et al., 2021). Furthermore, RL based controllers can learn from experience and improve their performance over time, leading to more efficient and robust spacecraft control. As a result, RL has the potential to enable more precise and autonomous spacecraft operations, which can lead to reduced costs and increased safety in space exploration and commercial applications.

DRL combines RL with the field of deep learning. It tackles problems by formulating them as discrete-time Markov Decision processes (MDPs), allowing the model's state and transitions to be described mathematically (White, Gass, & Harris, 2001). At each timestep, the agent interacts with its environment and receives feedback in the form of state observations and a reward that assesses the suitability of the current state, as shown in Fig. 2. The agent then creates an action signal consisting of the required thrusts and torques to adjust the state as it sees fit. The agent learns by repeating this process and adjusting the weights and biases in its neural networks to achieve the maximum reward.

The controller proposed in this paper uses an HDRL framework, this consists of multiple DRL algorithms with independent policies operating at different timescales and feeding sub-goals down through the hierarchy. These algorithms use multiple policies which speeds up the learning process and allows for learning at different time resolutions, increasing their sample efficiency (Levy, Konidaris, Platt and Saenko, 2017; Levy, Platt and Saenko, 2017). Of the existing HDRL models a DDPG based Hierarchical Actor-Critic framework has been chosen as the model architecture. The HAC architecture uses separate actors and critics for each agent and sub-agent, it also utilises hindsight experience replay (HER) to allow for complicated behaviours to be learned using sparse or binary rewards. Each layer of the HAC hierarchy is tasked with solving a subproblem and producing subgoals for the level below it. The first or highest layer receives the system state and overall goal state as input, the last or lowest layer produces the actions for the satellite to execute. The main advantage of using an HAC architecture is that the policies used operate on shorter timescales allowing for faster learning (Levy, Platt et al., 2017).

2.3. Attitude testing platforms

The goal of any attitude testing platform is to create a frictionless environment that mimics the conditions that the control system in question would experience. There are multiple ways this can be done; one way is to place the controller within a gyroscope giving it the freedom of movement it requires. An example of this method is from Hadi (2015), where three frames are placed within one another and connected using ball bearings to create a gyroscopic platform that is both low-cost and effective. The main drawback of this method is that any friction present in the bearings may cause unwanted disturbances in the system.

Another possible method is using a tabletop air bearing. These are platforms placed atop or connected to a hemisphere, also known as a rotor. This rotor rests on a thin film of air between itself and the stator below it; the stator is a cup-shaped part that generally has many channels running through it allowing for compressed air to escape and create a layer for the rotor to float on. For tabletop air bearings to function correctly the centre of gravity of the platform must remain close to and in line with its centre of rotation; this can either be done through careful design as is the case in Long (2014) or through the use of an automatic balancing system built into the tabletop as seen in Newton (2021).

The final and most common way to build an attitude control testing platform is by placing the object of study within a hemispherical or spherical air bearing. This method utilises the same air-bearing mechanism used in the tabletop devices but instead of having a platform resting on top of a hemisphere, the controller is placed within the hemisphere or sphere that itself acts as a rotor. Having the controller secured within the hemisphere allows for a simpler design and a bigger range of motion; it is for these reasons that a hemispherical air bearing has been selected as the structure for the attitude control testing platform.

2.4. Rendezvous testing platforms

The Rendezvous and Docking (RvD) stage of a spacecraft's mission is generally the most challenging phase of the mission. Careful testing and analysis must take place before attempting to utilise new technologies in this section of the mission. In order to perform HIL testing on an RvD controller a space environment setup must be used.

It is possible to use a planar air bearing for RvD testing such as is seen in Cookson (2019) and Santaguida and Zhu (2023). This method of testing satellite rendezvous operations requires two spacecraft simulators with planar air-bearing "feet" to allow an air cushion between themselves and a flat plane such as a granite table. These spacecraft simulators contain compressed air canisters and regulators to ensure a smooth and consistent output of compressed air is passed through the air bearings. In addition, planar testing platforms can only reproduce 3 degrees of freedom limiting the testing capabilities of the platform as a whole. These types of testing platforms also take up a large amount of space to allow for freedom of movement and require a large flat surface such as a granite table which can be difficult to acquire and install.

An alternative method of testing a satellite's RvD system is by using robotic arms to simulate the movement of the spacecraft in a space environment. Unlike the planar air bearing method this method is capable of giving the spacecraft six degrees of freedom allowing the freedom of movement for a complete test of the spacecraft RvD system. The most well-known example of such a system is the European Proximity Operations Simulator (EPOS 2.0) facility located at the DLR Space Operations and Astronaut Training institution in Oberpfaffenhofen (Center, 2023). This facility consists of two Kuka industrial robots, one of which is mounted on a rail spanning 25 m, allowing it to approach the other as if they were two satellites in a rendezvous scenario. Both robots have six degrees of freedom and are highly accurate in their positioning capabilities allowing for sub-millimeter range precision along the 25 metres of rail. Further, a high-performance solar simulator is present in the facility allowing for realistic ambient lighting for any optical sensors that may require it.

Another example of a lower cost robotic arm-based testing platform can be found in Gavrilovich (2016). In this study, the author develops a novel "AirBall" Satellite simulator; this includes a mock CubeSat placed within a partial spherical air bearing to allow for frictionless movement. The AirBall is fully contained within an outer sphere with the mock cubesat contained within an inner sphere with air-bearing pucks located between them to provide the cushioning layer of air. The AirBall provides rotational degrees of freedom to the mock cubesat, in order to provide translational degrees of freedom the AirBall is attached to the end of an Adept Viper s650, a six DoF articulated robot.

3. Reinforcement learning formalism

3.1. Deep reinforcement learning

The idea behind reinforcement learning is to enable an intelligent agent to learn and make decisions through interactive experiences

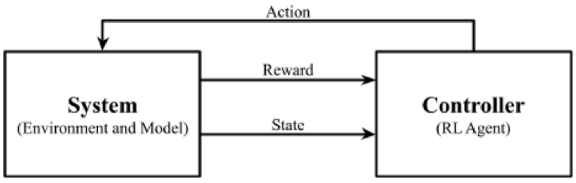


Fig. 2. A generic RL model.

within an environment. Unlike other machine learning methods that rely on labelled data, reinforcement learning operates in an interactive setting, where the agent optimises its behaviour over time. The underlying principle is that the agent learns by trial and error, exploring different actions and observing the outcomes they produce. Through this iterative process, the agent gradually builds a model of the environment and discovers actions that lead to the most favourable outcomes. Ultimately, the goal of the agent is to learn an optimal policy that maximises the expected cumulative reward, allowing it to make intelligent decisions and adapt to dynamic environments.

The reinforcement learning process revolves around a cyclic sequence of observation, action, and reward. At each time step, the agent observes the current state of the environment and selects an action based on this observation. Subsequently, the environment transitions to a new state, and the agent receives a numerical reward signal that indicates the desirability of its action. This feedback guides the agent's learning process. By repeating this cycle, the agent explores different states and actions, gradually updating its policy to enhance its decision-making capabilities.

At the heart of reinforcement learning lies the mathematical framework known as a Markov decision process. This framework formalises the sequential decision-making problem by characterising the environment as a set of states, actions, transition probabilities, and rewards. This framework assumes the Markov property, which states that the future state relies solely on the current state and action, disregarding any history leading up to it. This simplifies the decision-making process, enabling the agent to make optimal choices based solely on the current state.

To assess the agent's performance and determine the desirability of a state, a reward function $r(s_t, a_t)$ assigns a numerical value that describes the agent's current performance. The cumulative discounted reward function is employed as a performance indicator to evaluate the long-term desirability of a state (Fujimoto, van Hoof, & Meger, 2018):

$$R_t = \sum_{i=t}^{T} \gamma^{i-t} r(s_i|a_i) \tag{1}$$

where $\gamma \in (0,1]$ is a discount factor which discounts expected future rewards in favour of current rewards. The RL agent operates by iteratively interacting with the environment, observing its consequences through the use of the reward function. It does this to optimise the policy function $\pi(a_t|s_t)$, which attempts to map observations to actions in order to maximise the expected reward.

An action-value function gives the expected reward r_t when the agent takes an action a_t in a given state s_t, while following a policy π:

$$Q^{\pi}(s_t, a_t) = E[R_t|s_t, a_t] \tag{2}$$

The policy and value functions are approximated through the use of artificial neural networks, which are composed of interconnected nodes or neurons, each with adjustable weights and biases that influence the strength of their connections (Wang, 2003). Through this neural network architecture, the agent can effectively approximate the policy and value functions, facilitating the decision-making and learning processes.

3.2. The DDPG and TD3 algorithms

For this paper the TD3 algorithm was used to develop the DRL agents. The TD3 algorithm is an improvement on the DDPG algorithm, as such, a brief description of this algorithm will be provided before stating the additions made to create the TD3 algorithm.

The DDPG algorithm is a well known model-free off-policy RL algorithm used to tackle problems with continuous action spaces. At its core, the DDPG algorithm uses the actor-critic method, which maintains separate actor and critic networks. The actor network determines the policy of the agent and the critic network serves as a value function approximator, estimating the expected return or Q-value associated with a given state–action pair (Lillicrap et al., 2015).

Fig. 3 describes the structure of a DDPG agent. The DDPG algorithm begins with the initialisation of the actor and critic networks with random weights; these networks are duplicated to produce the actor and critic target networks, used to stabilise the learning process by reducing learning variance. At each iteration, the agent observes the current state of the environment, inputs this observation to the actor network which then produces an action for the agent to take. The agent then interacts with the environment by executing the selected action and receiving the next state and reward value. To improve learning efficiency, the observed transitions (s_t, a_t, r_t, s_{t+1}), a tuple of the current state, action, reward, and next state, are stored in a replay buffer for experience replay. This buffer enables the agent to sample mini-batches of transitions, breaking the correlation between consecutive experiences and being used to update the actor and critic networks. Updating the neural network refers to adjusting or modifying the weights and biases within the network during training to improve its predictions.

The critic network works to estimate the expected return for a given state–action pair, also known as the Q-value. The critic network is updated by minimising the mean squared Bellman error (MSBE) between the predicted Q-value and the target Q-value, this is shown in Fig. 3 as the update signal from the "MSBE Loss Function" block. The target Q-value is computed using the target critic network and the next state, following the Bellman equation:

$$Q^{\pi}(a_t, s_t) = r_t + \gamma E[Q^{\pi}(a_{t+1}, s_{t+1})] \tag{3}$$

The actor network aims to improve the agent's policy based on the observed states. It updates its weights by maximising the expected return, as estimated by the Q-value produced by the critic network. The actor network update is performed by following the policy gradient ascent direction. The actor network weights, θ, and $J(\theta)$, at time t are determined by the policy gradient equation:

$$\nabla_{\theta} J(\theta) = E[\nabla_{a_t} Q^{\pi}(a_t, s_t) \nabla_{\theta} \pi_{\theta}(s)] \tag{4}$$

To stabilise the learning process, the target networks are given a 'soft' update by blending the weights of the current actor and critic networks with a fraction of the target networks' weights, this is visualised in Fig. 3 by the update signals send from the current networks to the target networks. By following the principles of the Bellman equation and policy gradient, DDPG allows the agent to iteratively refine its decision-making process and optimise its policy. With each iteration, the agent's ability to make informed and adaptive decisions improves, leading to better performance in continuous control tasks.

While the DDPG algorithm has shown promising results in various continuous control tasks, it also faces certain challenges. The algorithm is prone to overestimate Q-values which can lead to unstable learning and suboptimal policies. Additionally, DDPG is sensitive to hyperparameter settings, such as learning rates and exploration noise levels; while choosing hyperparameter values is crucial for achieving effective learning and stable convergence it can be time consuming to find the appropriate values. Lastly, DDPG is sample inefficient, requiring a large number of interactions with the environment to learn effective policies. The need for extensive data collection can limit its applicability in scenarios where interactions are costly or time-consuming.

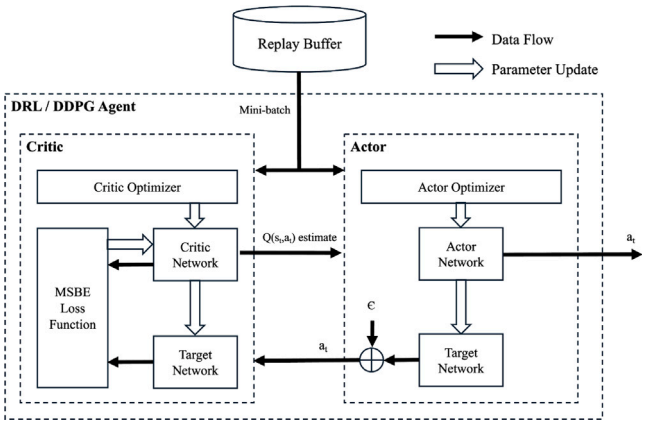


Fig. 3. The structure of a DDPG agent.

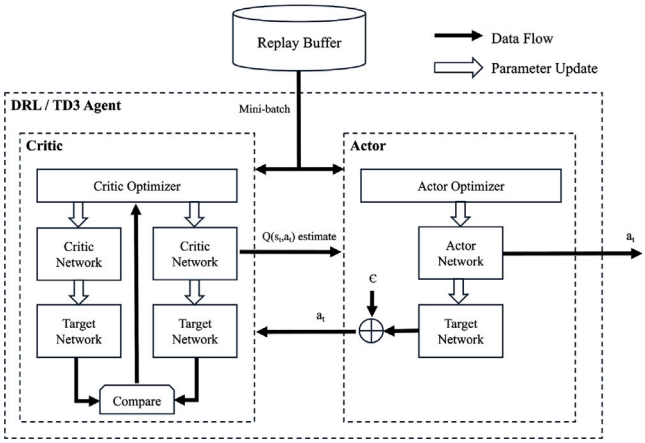


Fig. 4. The structure of a TD3 agent.

Fig. 4 describes the structure of a TD3 agent. The TD3 algorithm is an extension of the DDPG algorithm that addresses some of its limitations, improving overall stability and performance. There are three major additions made to DDPG to create the TD3 algorithm. These are delayed policy and target network updates, twin critic networks and target policy smoothing.

The use of twin critics instead of a single critic network helps reduce overestimation bias in the Q-value estimation. By having two separate critic networks that estimate Q-values independently, TD3 can select the minimum Q-value between the two as the target for updating the actor network. This minimisation process directly addresses the overestimation issue identified in standard Q-learning methods, as shown in Fujimoto et al. (2018), where the inherent noise introduced by function approximation can lead to an overestimation of the Q-values. By taking the minimum of the two Q-values, TD3 introduces a conservative bias that acts as a counterbalance to the overestimation problem, ultimately leading to more accurate value estimates.

Furthermore, instead of updating the policy and target networks every iteration, TD3 updates them less frequently. This delayed update mechanism ensures that the networks' weights remain stable and prevents frequent and large fluctuations. By introducing this delay, TD3 effectively reduces the risk of destabilising the learning process, as rapid updates could lead to feedback loops where errors propagate between the actor and critic too quickly, causing instability in the policy learning process (Dankwa & Zheng, 2019).

To further mitigate the overestimation of Q-values, TD3 also employs target policy smoothing. This involves adding exploration noise to the target actions during training. By smoothing out the target actions, TD3 encourages the agent to explore a wider range of actions and helps prevent the policy from getting stuck in local optima. The TD3 algorithm also maintains a separate target actor network that is updated less frequently; this network is used for generating target actions during the Q-value estimation process. These combined improvements – twin critics, delayed updates, and target policy smoothing – contribute to the overall stability and accuracy of the value estimation. It is for these reasons that we have selected the TD3 algorithm to construct the DRL agents tested in this paper.

3.3. The HAC framework

Despite the promise of DRL algorithms like DDPG and TD3 they can still face challenges when applied to increasingly complex tasks, specifically when encountering high-dimensional state and action spaces, long time horizons, and exploration in large solution spaces. HDRL has presented itself as a way to address these challenges. The HDRL controller used in this paper will follow a HAC framework. The structure of a HAC agent, as illustrated in Fig. 7, consists of multiple DDPG agents organised in a hierarchy. Each level in the hierarchy operates as an independent DDPG agent, with higher-level agents selecting subgoals for lower-level agents to achieve, enabling efficient exploration and exploitation across different time scales. The update mechanisms within this hierarchical framework rely on individual DDPG agents that are updated using standard off-policy learning algorithms as shown in Fig. 3.

One of the main components of the HAC structure is the use of Universal Value Function Approximators (UVFA) (Schaul, Horgan, Gregor, & Silver, 2015). A UVFA modifies the Q-function $Q^{\pi}(s_t, a_t)$ by including a goal state g_t, which is the state an agent should be working towards, allowing the Q-function to map states to actions with respect to a given goal state $Q^{\pi}(s_t, a_t, g_t) = E[R_t|s_t, a_t, g_t]$. It is useful to define a goal since goals are often hierarchical in nature and can be broken down into subgoals to accelerate the learning process. It also removes the need for a reward function as a reward can be determined by whether the agent achieves the goal set for it or not, reducing the reward to a binary outcome. The main drawback of using UVFA is that it is less effective when dealing with complex problems with sparse rewards (Levy, Konidaris et al., 2017).

To compensate for the drawbacks of UVFA the concept of Hindsight Experience Replay is introduced as it allows the agent to learn goal-based policies significantly faster, especially when considering sparse rewards (Andrychowicz et al., 2017). HER allows the agent to learn from unsuccessful experiences by reframing them as successful ones, it achieves this by replaying an unsuccessful episode as a successful one with a different goal selected based on the final state of the episode. By doing so, the agent can learn from the failed episode as if it had succeeded, which can speed up learning and improve sample efficiency. It is implemented by saving a copy of the experience $(s_t, a_t, r_t, s_{t+1}, g_t)$ with the goal replaced by the one achieved in hindsight and the reward recalculated based on this modified goal.

Another distinctive feature of HDRL is the introduction of temporal abstraction. This enables agents to operate at different time scales, organising actions into multiple levels of varying abstraction and deconstructing complex tasks into a hierarchy of subtasks. Each subtask represents a high-level action that can extend over a period of time, these can either be selected before training or learned independently like in the case of HAC. This hierarchical structure resembles a decision tree, with higher-level subtasks selecting lower-level subtasks until primitive actions are executed. Furthermore, HDRL allows for the reuse of learned subtasks across different parts of the state space and even in entirely different tasks. Once an agent has acquired a set of valuable high-level tasks, it can apply them to new tasks, expediting the learning process and often requiring fewer samples to achieve similar results. An additional advantage of HDRL comes from its ability to enable exploration at multiple levels. High-level tasks allow agents to explore and discover promising subgoals or macro-actions, expediting the learning process.

4. Problem statement

4.1. Spacecraft attitude dynamics

A scalar first quaternion representation was chosen to model the satellite attitude dynamics. The attitude error at time t is calculated by:

$$q_e = q_d^{-1} \otimes q_t \tag{5}$$

where q_d is the desired final orientation and q_t is the quaternion at time t. The spacecraft's attitude can be propagated through time by integrating the kinematic equation at each timestep (Markley & Crassidis, 2014):

$$\dot{q}_e = \frac{1}{2}\Omega q_e \tag{6}$$

Ω is the augmented cross-product matrix, this matrix is constructed using the angular rate vector of the spacecraft ω_s:

$$\Omega = \begin{bmatrix} 0 & -\omega_s^T \\ \omega_s & -\omega_s^\times \end{bmatrix} \tag{7}$$

And the angular rate cross-product matrix $\omega_s^\times$ is:

$$\omega_s^\times = \begin{bmatrix} 0 & -\omega_{s,z} & \omega_{s,y} \\ \omega_{s,z} & 0 & -\omega_{s,x} \\ \omega_{s,y} & \omega_{s,x} & 0 \end{bmatrix} \tag{8}$$

The dynamics of a spacecraft with three reaction wheels as its actuators can be modelled using Euler's rotational motion equation for rigid bodies (Hilton, 1986; Wertz, 1990):

$$\frac{dL}{dt} = T_d - \omega_s^\times L \tag{9}$$

where L is the total angular momentum of the spacecraft and T_d is the disturbance torque acting on it. If L_{rw} is defined as the angular momentum of the reaction wheels and I the spacecraft's moment of inertia matrix then L can be written as:

$$L = I\omega_s + L_{\text{rw}} \tag{10}$$

Substituting this into Eq. (9) gives:

$$\frac{d}{dt}(I\omega_s + L_{\text{rw}}) = T_d - \omega_s^\times(I\omega_s + L_{\text{rw}}) \tag{11}$$

$$\dot{\omega}_s = I^{-1}(T_d - \dot{L}_{\text{rw}} - \omega_s^\times(T\omega_s + L_{\text{rw}})) \tag{12}$$

4.2. Relative orbital dynamics

The Clohessy–Wiltshire (CW) equations have been chosen to model the relative motion between the target and chaser spacecraft. They provide a linearised solution to the relative dynamics of two spacecraft in close proximity, assuming that they follow a circular orbit. The equations are given in the Hill reference frame where the origin is on the target spacecraft, the x-axis points radially away from Earth's centre, the y-axis is parallel to the orbital momentum vector and points in the direction of the orbit, and the z-axis completes the right-handed coordinate system.

The CW equations can be expressed as a set of linear, time-invariant, and coupled second-order differential equations:

$$\ddot{x} = 3n^2x - 2n\dot{y} + \frac{F_x}{m} \tag{13}$$

$$\ddot{y} = -2n\dot{x} + \frac{F_y}{m} \tag{14}$$

$$\ddot{z} = -n^2z + \frac{F_z}{m} \tag{15}$$

In these equations, x, y, and z represent the relative positions of the target with respect to the chaser. $\mathbf{F} = [F_x, F_y, F_z]$ is some force applied onto the chaser and m is its mass. The variable n denotes the mean orbital motion of the reference spacecraft, which is related to the gravitational constant, G, the mass of the Earth, M, and the semi-major axis of the target's orbit, a_t, through the relation $n = \sqrt{GM/a_t^3}$.

If we define the system's state as $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^T$ then the CW equations can be described by the linear time-invariant (LTI) system:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t) = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \tag{16}$$

where $\mathbf{u} = \frac{\mathbf{F}}{m}$, the state matrix $\mathbf{A}$ and the input matrix $\mathbf{B}$ are given by:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{bmatrix} \tag{17}$$

$$\mathbf{B} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{18}$$

The closed form solutions of these equations can be given in matrix form:

$$\mathbf{x}(t) = \begin{bmatrix} \mathbf{\Phi_{rr}} & \mathbf{\Phi_{rv}} \\ \mathbf{\Phi_{vr}} & \mathbf{\Phi_{vv}} \end{bmatrix} \mathbf{x}(0) \tag{19}$$

where:

$$\mathbf{\Phi_{rr}} = \begin{bmatrix} 4 - 3\cos nt & 0 & 0 \\ 6(\sin nt - nt) & 1 & 0 \\ 0 & 0 & \cos nt \end{bmatrix}$$

$$\mathbf{\Phi_{rv}} = \frac{1}{n}\begin{bmatrix} \sin nt & 2 - 2\cos nt & 0 \\ 2\cos nt - 2 & 4\sin nt - 3nt & 0 \\ 0 & 0 & \sin nt \end{bmatrix}$$

$$\mathbf{\Phi_{vr}} = \begin{bmatrix} 3n\sin nt & 0 & 0 \\ 6n(\cos nt - 1) & 0 & 0 \\ 0 & 0 & -n\sin nt \end{bmatrix}$$

$$\mathbf{\Phi_{vv}} = \begin{bmatrix} \cos nt & 2\sin nt & 0 \\ -2\sin nt & 4\cos nt - 3 & 0 \\ 0 & 0 & \cos nt \end{bmatrix}$$

4.3. Environmental disturbances

Spacecraft encounter a wide range of environmental disturbances that can be detrimental to their performance and significantly impact mission success. It is important to consider these disturbances in order to develop sufficiently robust control strategies. The four types of environmental disturbances that will be considered are; gravitational perturbations, atmospheric drag, solar radiation pressure, and magnetic field disturbances.

Gravity gradient torques are caused by the difference in gravitational forces along the length of the satellite. This will cause the satellite to experience a torque that will tend to align the axis of least inertia to the direction of the gravitational field. This can be calculated using (Roberson, 1964):

$$T_{\text{grav}} = \frac{3\mu}{R^3}\vec{u_e} \times I\vec{u_e} \tag{20}$$

where T_{grav} is the gravity gradient torque, μ is the geocentric gravitational constant, R is the distance from the Earth's centre to the satellite, and $\vec{u_e}$ is a unit vector pointing towards nadir.

Aerodynamic torque is highly dependent on the altitude of the satellite; in the lower sections of LEO the atmosphere is dense enough to cause drag on a satellite resulting in orbital decay and angular moments. The atmosphere at these heights follows a rarefied free molecular flow regime rather than a continuum flow regime; this means

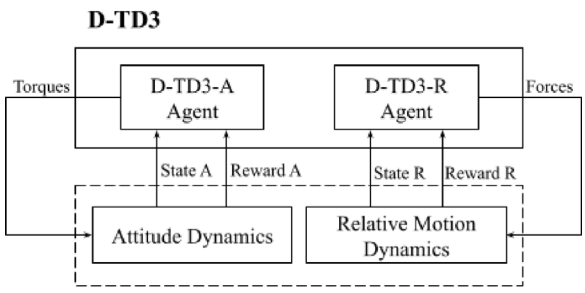


Fig. 5. The D-TD3 agent's block diagram.

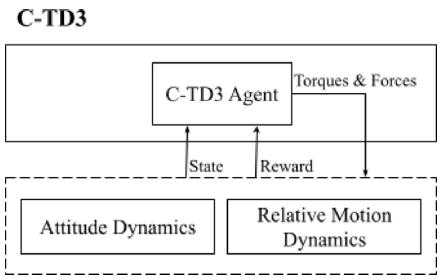


Fig. 6. The C-TD3 agent's block diagram.

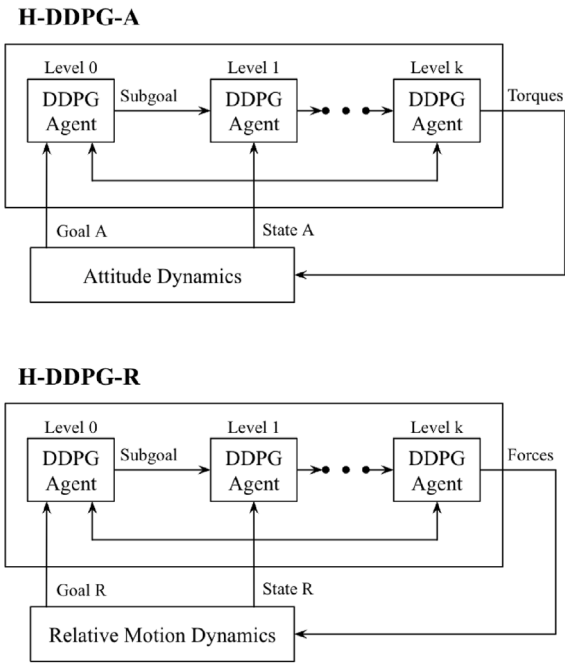


Fig. 7. The H-DDPG agent's block diagrams.

that the drag can be estimated by Markley and Crassidis (2014) and Regan and Anandakrishnan (1993):

$$T_{\text{areo}} = \frac{1}{2}\rho V^2 C_d A(\vec{u_v} \times \vec{s_{cp}}) \tag{21}$$

where T_{areo} is the torque on the satellite caused by aerodynamic drag, ρ is the air density, V is the satellite velocity, C_d is the drag coefficient, A is the area affected by drag, u_v is the unit velocity vector, and s_{cp} is the vector from the centre of pressure to the centre of mass.

Solar radiation pressure is a phenomenon resulting from the impact of photons onto space objects. As sunlight strikes a satellite, the photons each exert a minute force on its surface, causing a subtle yet persistent force. While this effect is generally quite small, it can influence the orbits of satellites over time. Generally the other sources of environmental disturbances dwarf that of solar radiation pressure when considering a LEO satellite.

Earth's magnetic field causes unwanted torques for LEO satellites that can affect its orientation and orbit if not resisted against. These torques are caused by fluctuations in earth's magnetic field interacting with the satellite's magnetic dipole. The strength of the magnetic field changes depending on the satellites position relative to earth, the magnetic field intensity B can be estimated by Aman, Arelhi, and Khan (2019):

$$B = \frac{B_0}{R^3}\sqrt{1 + \sin\gamma^2} \tag{22}$$

where B_0 is the magnetic field on earth's surface at the equator, R is the satellite's orbital height, and γ is the magnetic latitude. The maximum magnetic disturbance torque the satellite could experience can be calculated by multiplying B by the satellite's residual dipole D:

$$T_{\text{mag}} = DB \tag{23}$$

5. Methodology

5.1. DRL agent structure and design

Two DRL based control systems have been selected for testing; A Decentralised TD3 control system (D-TD3) and a Centralised TD3 control system (C-TD3). The D-TD3 control system consists of two separate TD3 agents; they are named D-TD3-R and D-TD3-A, responsible for solving the rendezvous and attitude control problems respectively. These agents function independently, with their own reward functions and observation spaces. The outputs of each agent consist of force or torque commands used to solve their respective control problem. The C-TD3 control system is made up of one combined TD3 agent that merges the reward functions and observation spaces of both D-TD3 agents creating a unified input to the singular centralised agent (see Figs. 5 and 6).

A generic 1U CubeSat in LEO has been selected as the chaser spacecraft, with a diagonal moment of inertia $I_{xx} = I_{yy} = I_{zz} = 0.001\,\text{kgm}^{-2}$ and a mass of 0.5 kg. The training environment is initialised by setting an initial orientation with an angular error between 10° and 90°, zero angular velocity, a small relative velocity, and a distance of around 100 m from the target measured parallel to the angular momentum vector of the target spacecraft. The agents observe the environment through the use of an observation state. The observation into attitude and rendezvous components; the attitude component consists of the spacecrafts angular rate $\omega = [\omega_x, \omega_y, \omega_z]$ and its error quaternion $q_e = [q_{e,w}, q_{e,x}, q_{e,y}, q_{e,z}]$. The rendezvous component consists of the spacecraft's relative position $\mathbf{r} = [r_x, r_y, r_z]$ and velocity $\mathbf{v} = [v_x, v_y, v_z]$ with respect to the target. These observation states are used separately for the agents that make up the D-TD3 controller or they can be combined for use in the C-TD3 controller. The force and torque commands are limited to 0.1N and 0.001 N m respectively.

5.2. Reward shaping

The goal of the control system is to bring the angular rate and angular error of the spacecraft below $1°/s$ and 5° respectively, and come within 5 m of the target satellite. The TD3 agents require a reward function to be designed to drive the agents to the desired goal states.

The reward used is split into two parts; the rendezvous and attitude components. At first both parts use binary rewards to push the spacecraft to the target while decreasing orientation error.

$$r_r = \begin{cases} 0.1 & \text{if } \|\mathbf{r}(t_i)\| < \|\mathbf{r}(t_{i-1})\| \\ -0.1 & \text{otherwise} \end{cases} \tag{24}$$

$$r_a = \begin{cases} 0.1 & \text{if } \dot{q}_{e,w} \geq 0 \\ -0.1 & \text{otherwise} \end{cases} \tag{25}$$

When the spacecraft reaches a relative distance of 25 m from the target the rendezvous reward switches to a continuous reward.

$$r_r = \exp\left(\frac{\text{distance from target}}{45}\right) \tag{26}$$

Similarly when the angular error of the agent is below 10° the attitude reward also becomes continuous

$$r_a = q_{e,w}^2 - q_{e,x}^2 - q_{e,y}^2 - q_{e,z}^2 \tag{27}$$

In addition, small bonus rewards are added to encourage certain behaviours; whenever the agent is either within 10 m of the target or achieves an angular error of less than 5° a bonus reward of 4 is added. There are also terminal rewards included in the model; if the angular rate exceeds 90°/s or the relative distance exceeds 250 m the episode is stopped prematurely and a large negative reward of −100 is applied to discourage the agent.

5.3. HDRL agent structure and design

The H-DDPG controller follows a similar structure to the D-TD3 controller, made up of two DDPG based HAC agents, named H-DDPG-A and H-DDPG-R, each consisting of multiple DDPG agents arranged hierarchically. The output of the first or highest level of the hierarchy is fed into the subsequent agent as a subgoal, this is repeated through all the levels of the hierarchy until the final level in which the command forces and torques are generated. A block diagram of this structure is presented in Fig. 7.

6. Simulation

6.1. DRL framework training and simulation

The training setup consists of a disturbance-free environment, with each training episode lasting a maximum of 2000 timesteps, where each timestep lasts 0.1 s, and the spacecraft dynamics are propagated every 0.01 s. The environment was created in Python using OpenAI's Gym API, which facilitates interactions between the environment and the DRL agents. A standard TD3 algorithm, as described in Section 3, was developed and implemented using the TensorFlow package.

During training, the TD3 agents interact with their environment, taking actions and receiving rewards, thus learning optimal policies over time. Specifically, the agent adjusts its policy by updating the actor and critic networks based on the rewards gained. These networks feature three fully connected hidden layers employing ReLU (Rectified Linear Unit) activation functions, where ReLU is defined as $f(x) = \max(0, x)$, outputting the input directly if positive, and zero otherwise. The weights and biases of the neural networks are updated using the Adam optimiser. The agent's training ceases when the total episode rewards plateau or the maximum achievable episode reward is reached. In this setup, the saved weights and biases from the fully trained models are used during inference in the testing phase.

Following the completion of the training phase, the trained agents were moved to a more realistic simulation environment, which incorporated disturbance forces as well as actuator noise, to better assess their performance in real-world conditions. Both frameworks will be tested with the same initial conditions to better compare them. The testing scenarios will have the agents start with a relative position error of $[10\text{ m}, 45\text{ m}, 100\text{ m}]$ and an angular error of $[5°, 30°, 10°]$.

The hyperparameters used during training were hand selected through an iterative process. The process began by selecting a sensible value for each hyperparameter based on the values commonly used for DRL benchmark problems (Ding et al., 2020). The learning rate, discount factor, and batch size were then fine-tuned by training multiple agents with different values and observing their training performance to determine the best-performing hyperparameters based on the improvement of the total episode rewards.

Table 2
Table of D-TD3 and C-TD3 training parameters.

Parameters	D-DT3 values	C-TD3 values
Learning rate	0.001	0.001
Batch size	10^6	10^6
Mini-batch size	512	512
Optimiser	Adam	Adam
Discount (γ)	0.995	0.995
FC Layer 1 Nodes	64	512
FC Layer 2 Nodes	128	1024

Table 3
Table of H-DDPG training parameters.

Parameters	H-DDPG values
Learning rate	0.0001
Batch size	10^7
Mini-batch size	1024
Optimiser	Adam
Discount (γ)	0.95
FC Layer 1 Nodes	64
FC Layer 2 Nodes	64

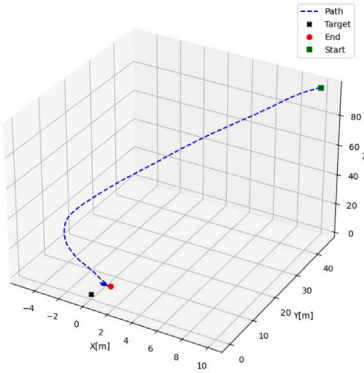
The learning rate was adjusted to strike a balance between fast convergence and stability, while the discount factor (γ) and batch size were selected based on their effectiveness in promoting learning progress. The number of nodes in the fully connected layers was reduced incrementally until performance degradation was observed, allowing the networks to remain computationally efficient without sacrificing accuracy. This approach ensured that the final hyperparameters, listed in Table 2, reflect those that consistently led to stable and efficient training across multiple trials.

The results of the simulations are displayed in Figs. 8 and 9. These results show that both control systems were able to achieve guidance and attitude control of the cubesat, bringing the spacecraft to rest at around 5 m from the target and achieving a pointing accuracy of 5°. The main difference between the two is the difference in time taken to correct the attitude error, with D-TD3-A taking less than 5 s and C-TD3 taking over 100 s. On the other hand, the results of the guidance control task are similar between both controllers, coming to rest at around 5 m from the target, with D-TD3-R taking a shorter path. When looking at the results of the C-TD3 controller it appears that it takes around the same time to solve both control problems despite the attitude control generally taking place at a shorter time scale. This indicates that using one agent to solve multiple control problems simultaneously would be inefficient and the controller would be better served by two agents working in tandem as seen in D-TD3.

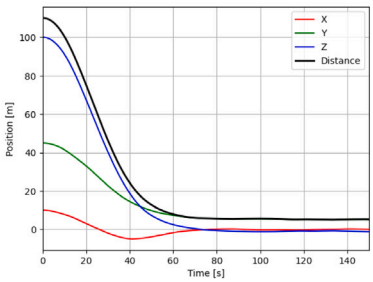
6.2. HDRL framework training and simulation

The training for the H-DDPG agents was conducted in the same environment as the D-TD3 agents, using the same network structure for each agent in the hierarchical framework. The HAC algorithm was adapted from Levy, Konidaris et al. (2017) and implemented using the PyTorch package. The training parameters shared by both H-DDPG agents, shown in Table 3, were selected through a process similar to that used for the TD3 agent hyperparameters. After the training phase, the H-DDPG agents were evaluated in the testing environment under the same initial conditions as those used for the D-TD3 and C-TD3 frameworks.

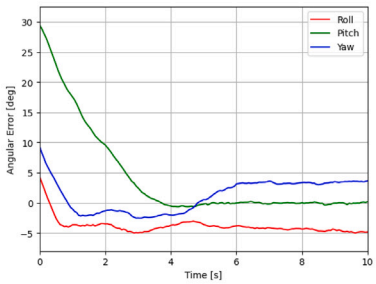
The simulation results for the H-DDPG control system, as shown in Fig. 10, demonstrate its capability to address both control problems ef-



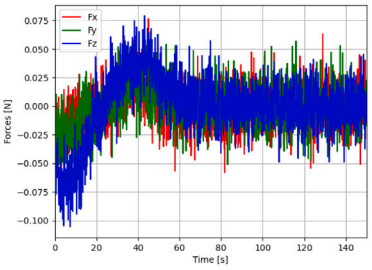
(a) D-TD3-R path.



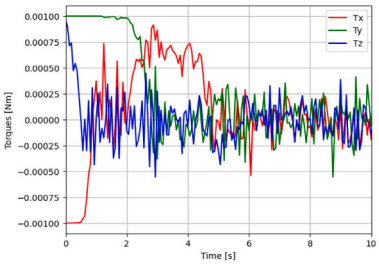
(b) D-TD3-R position error.



(c) D-TD3-A angular error.

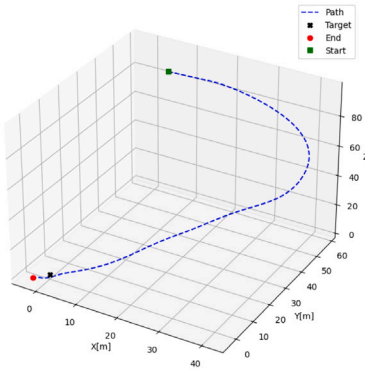


(d) D-TD3-R control forces.

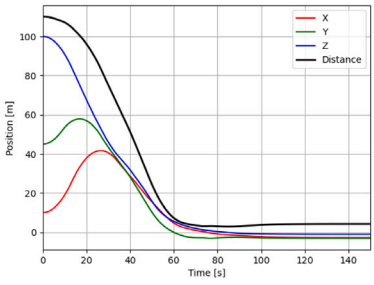


(e) D-TD3-A control torques.

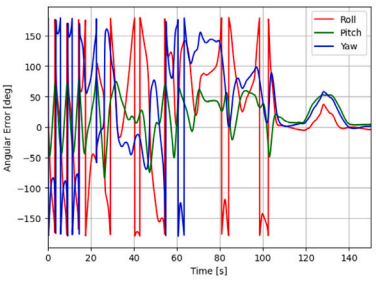
Fig. 8. Simulation results for the D-TD3 control system.



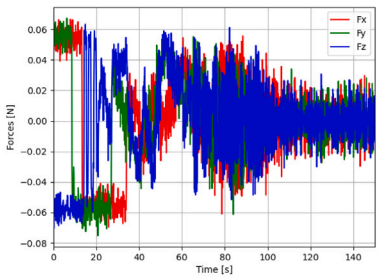
(a) C-TD3 path.



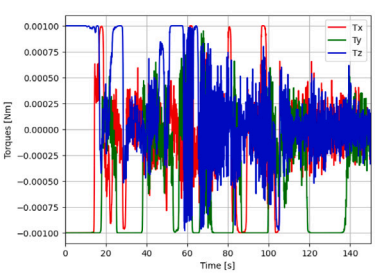
(b) C-TD3 position error.



(c) C-TD3 angular error.



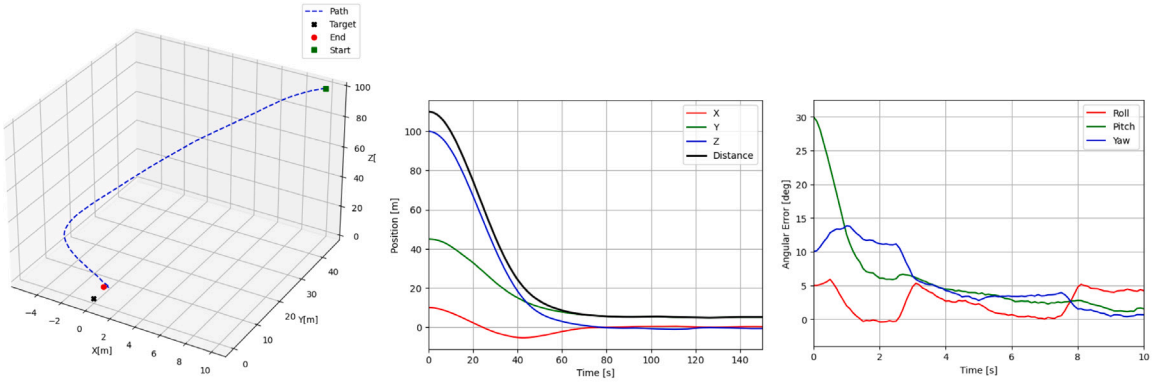
(d) C-TD3 control forces.



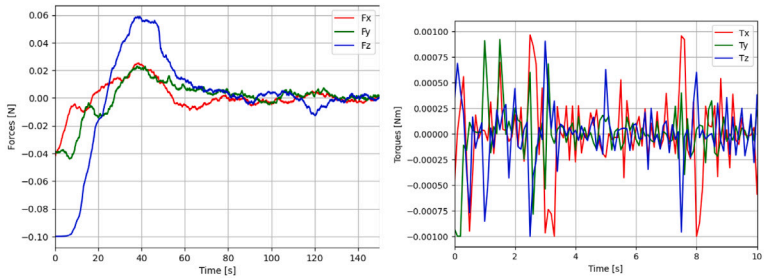
(e) C-TD3 control torques.

Fig. 9. Simulation results for the C-TD3 control system.

fectively, bringing the attitude and position errors within the prescribed tolerance levels. Although the attitude agent, H-DDPG-A, does not achieve attitude correction as smoothly as the D-TD3 agent, it reaches the desired goal in a comparable timeframe. Notably, the H-DDPG-A exhibits greater efficiency in terms of command torque, requiring less torque than the D-TD3 agent and showing less chattering. Similarly, while both agents perform comparably in the guidance problem, the H-DDPG-R agent commands substantially less control force and exhibits much less chattering compared to the D-TD3 agent, indicating superior control efficiency and stability in this case.

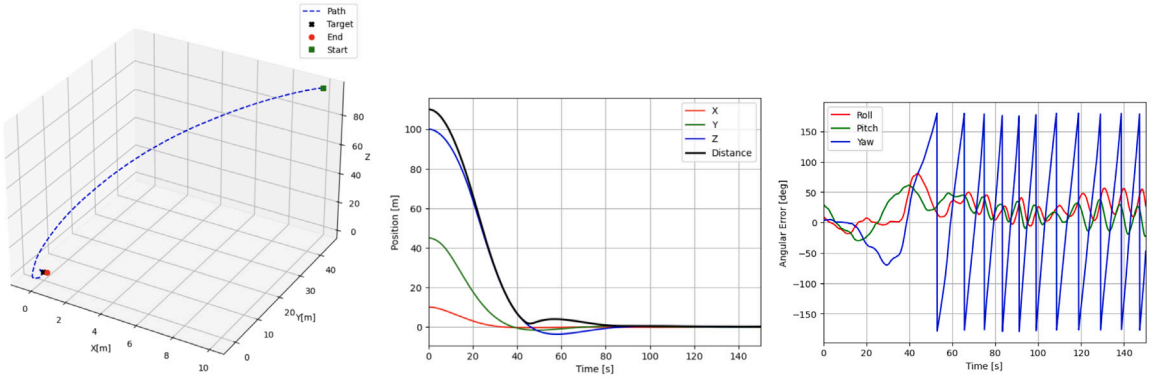


(a) H-DDPG path. (b) H-DDPG position error. (c) H-DDPG angular error.

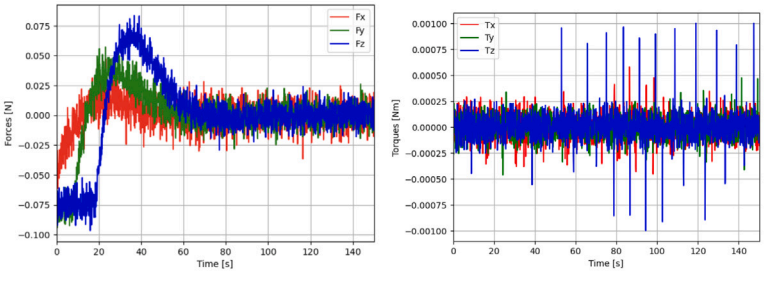


(d) H-DDPG control forces. (e) H-DDPG control torques.

Fig. 10. Simulation results for the H-DDPG control system.



(a) PD path. (b) PD position error. (c) PD angular error.



(d) PD control forces. (e) PD control torques.

Fig. 11. Simulation results for the classical PD control system.

To provide a baseline comparison, a classical proportional-derivative (PD) controller was also simulated under the same conditions as the H-DDPG agents, as shown in Fig. 11. The PD controller managed to correct its relative position error within the same timeframe as the H-DDPG-R agent and exhibited a lower final error. However, this came at the cost of significant chattering in the commanded forces, similar to the D-TD3 agent. In terms of attitude correction, the PD controller was unable to stabilise the satellite allowing it to tumble on its yaw axis while the roll and pitch angles oscillate between 0° and 50°. A possible explanation for this could be the presence of external disturbances and inherent limitations within the actuator system, such as nonlinearities or torque constraints in the reaction wheels. These

Table 4
Design parameters for the reaction wheel.

Parameter	Value
r_{disk} [mm]	18.75
r_{ring} [mm]	25.0
h_{disk} [mm]	2.5
h_{ring} [mm]	4.5
m_{disk} [kg]	0.022
m_{ring} [kg]	0.031

factors likely impeded the controller's ability to generate sufficient corrective torques, leading to instability and oscillations.

7. Hardware-in-the-loop testing

This section of the paper is subdivided into three parts each of which describes an independent part of the HIL testing platform. When all three parts come together they provide a testbed for simultaneous testing of a guidance and attitude control system. The section begins with a description of the design and creation of a mock CubeSat used for hardware testing. It will be bound by the same shape and weight constraints as a 1U CubeSat and should be able to sense changes in its position and orientation. Next, the attitude dynamics testing platform is introduced; this is designed to contain the mock Cubesat and provide a frictionless environment for the CubeSat to propagate its attitude freely. Finally, the rendezvous testing mission simulator is described; it will allow the CubeSat to move freely in 3D space through the use of a robotic arm.

7.1. The mock CubeSat setup

The mock CubeSat is designed to emulate the dimensions and functionalities of a standard 1U CubeSat; it serves as a critical element in the hardware testing phase. It should be capable of autonomously sensing its own orientation and transmitting this data to the integrated controller, allowing the CubeSat to react to changes in its attitude and proceed attitude control testing. In order to accomplish these goals, an Inertial Measurement Unit (IMU) capable of accurately sensing and relaying the details of the CubeSat's attitude is necessary. Alongside this, an actuation mechanism is needed to allow for the CubeSat to make adjustments to its attitude according to the instructions of the onboard control system.

The choice of the actuator plays a significant role in executing the attitude adjustments commanded by the control system. Commonly employed actuator systems include thrusters, magnetorquers, and reaction wheels. Thrusters are usually simulated using compressed air canisters housed within the CubeSat dispensing controlled amounts of compressed air as necessary to control the attitude. The main drawback of using thrusters is the space requirement that the compressed air canisters would take and the added difficulty of refilling or replacing the compressed air as needed. In order to use magnetorquers a Helmholtz cage must be constructed to cancel out Earth's electromagnetic field and simulate the geomagnetic field a satellite would experience in orbit. Finally, reaction wheels, also known as momentum wheels, operate on the principle of the conservation of angular momentum. They control the orientation of a satellite by rotating and alternating the spacecraft's rotational dynamics to provide correctional torques. For this research reaction wheels were chosen as the primary actuator of the satellite with one being placed on each of three perpendicular faces of the CubeSat to allow for 3-axis control.

There are a few design requirements that must be considered; all the components must fit within the 10 × 10x10 cm shape of a 1U CubeSat, and have a maximum overall weight of 1 kg. To meet these stringent design requirements, the CubeSat's internal architecture is crafted with a focus on optimal space utilisation and efficient power management. With this in mind, the components selected to construct the simulator are as follows:

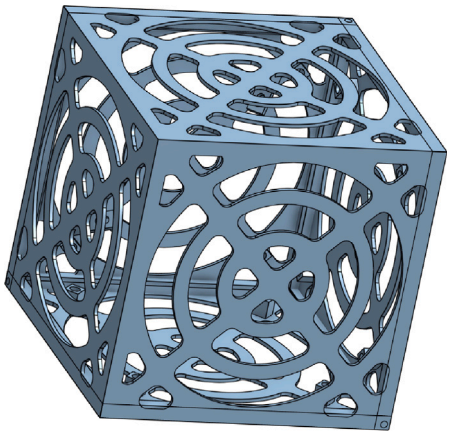


Fig. 12. The external structure of the mock CubeSat.

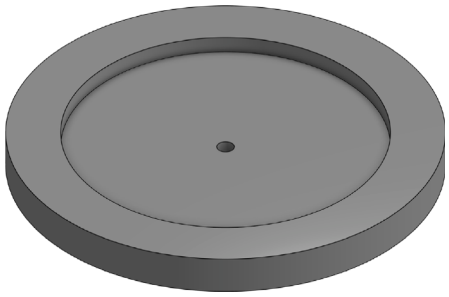


Fig. 13. The design of the reaction wheel.

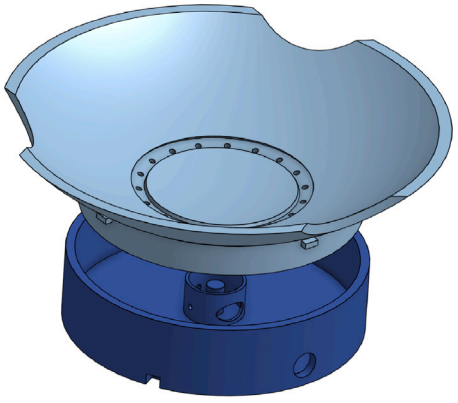


Fig. 14. The two piece stator design.

- The CubeSat's exterior structure is 3D printed in order to maintain a balance between being lightweight and rigid. It is printed in two parts to allow for the fitting of internal components and giving the option to replace or change internal components as needed. An image of this housing can be seen in Fig. 12.
- A 3S 11.1 V LiPo battery was chosen to power the electronics within the CubeSat due to its small size and having a high energy density.
- A 6-DoF MPU-6050 has been selected as the IMU for the CubeSat, it contains both a 3-axis gyroscope and a 3-axis accelerometer. This allows for the CubeSat to determine changes in its position and orientation making it an essential component for both testing platforms.
- An ESP32 Microcontroller (Systems, 2023) will contain the HDRL model, gather sensor data from the IMU, and transmit that data to

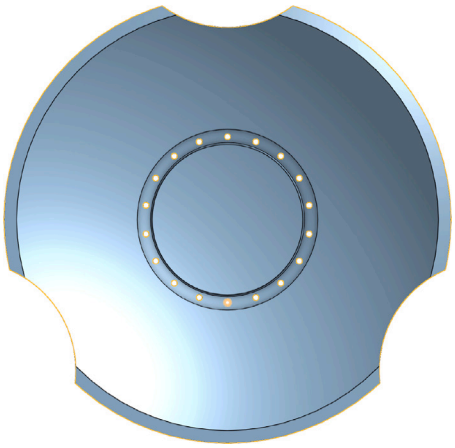


Fig. 15. Top view of the stator.

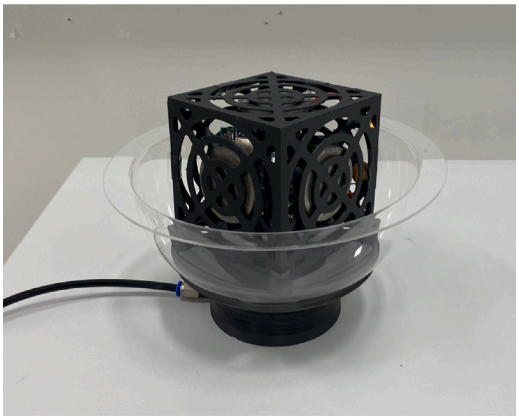


Fig. 16. The completed attitude dynamics testing platform.



Fig. 17. The combined attitude dynamics and rendezvous testing platform.

the motors in the case of attitude testing or to a central computer for the case of rendezvous testing. It is a low-cost low-power microcontroller with both Wi-Fi and Bluetooth capabilities.

- Three Faulhaber Brushless DC Motors with integrated speed controllers (Faulhaber, 2024) will be used to rotate the reaction wheels; these motors have been chosen due to their small size and adequate torque limits. In addition, they can be controlled directly using PWM signals, as they contain integrated speed controllers, removing the need to include external speed controllers that would take up the limited space within the CubeSat.
- The three reaction wheels have been manufactured in-house, they have a simple disc design with a raised edge where most of the mass is concentrated, this design allows for a lower total mass while maintaining a high level of momentum, this design can be seen in Fig. 13.

The design of the reaction wheels had to be carefully considered to ensure adequate actuation is possible. The main considerations for the design is its volume and mass. The reaction wheels in this experiment weighs 53 g each and have an outer diameter of 50 mm with a maximum height of 7 mm. This ensures that the reaction wheels can fit within the CubeSat while still providing appropriate actuation. Stainless steel has been chosen as the material for the reaction wheels due to its high density of $8 * 10^3$ kg/mm^2. The design details of the reaction wheel can be viewed in Table 4.

With this information it is possible to calculate the reaction wheel's moment of inertia matrix I_{total} as follows:

$$I_{\text{disk}} = \frac{m_{\text{disk}} r_{\text{disk}}^2}{2} = 3.87 \cdot 10^{-6} \tag{28}$$

$$I_{\text{ring}} = \frac{m_{\text{ring}} (r_{\text{ring}}^2 + r_{\text{disk}}^2)}{2} = 6.30 \cdot 10^{-4} \tag{29}$$

$$I_{\text{total}} = I_{\text{disk}} + I_{\text{ring}} = 6.34 \text{ kg/m}^2 \tag{30}$$

The maximum angular momentum $L_{\text{rw, max}}$ can be found by taking into account the maximum speed of the motor, 7000 rpm in this case:

$$L_{\text{rw, max}} = I_{\text{total}} \omega_{\text{max}} = 6.34 \cdot 10^{-4} \cdot 7000 \cdot \frac{2\pi}{360} = 0.077 \text{ kg m}^2 \text{ s}^{-1} \tag{31}$$

The time t it will take for the reaction wheel to rotate the CubeSat θ degrees can be estimated using the CubeSat's maximum moment of inertia I_{max} (Oland & Schlanbusch, 2009):

$$t = \frac{2 \cdot \theta \cdot I_{\text{max}}}{L_{\text{rw, max}}} = \frac{2 \cdot 90 \cdot 0.00167}{0.077} = 3.896 \text{ s} \tag{32}$$

This means that it will take a minimum of 3.896 s for the reaction wheel to rotate the CubeSat 90 degrees.

The controllers output the control torque, this must be transformed into a PWM signal that can be sent to the motors speeding them up or slowing them down depending on the direction of the torque. To accomplish this we can use the relationship between torque and the rate of change of angular momentum to determine the required change in angular velocity of the reaction wheels $\Delta\omega_{\text{rw}}$ to produce the control torque T_c commanded by the controller (Krishna et al., 2018):

$$T_c = -T_{\text{rw}} \tag{33}$$

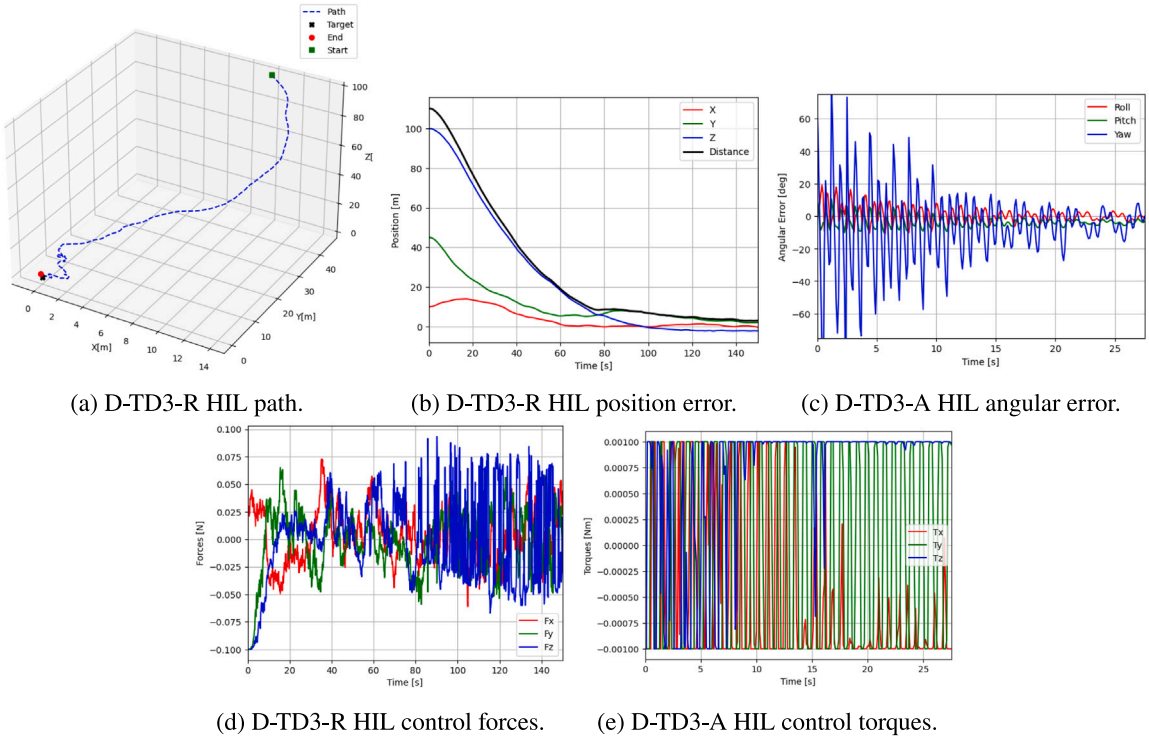
$$T_c = -\frac{d}{dt} L_{\text{rw}} \tag{34}$$

$$T_c = -\frac{d}{dt} (I_{\text{rw}} \omega_{\text{rw}}) \tag{35}$$

$$T_c = -\frac{I_{\text{rw}} \Delta\omega_{\text{rw}}}{\Delta t} \tag{36}$$

$$\Delta\omega_{\text{rw}} = -\frac{T_c \Delta t}{I_{\text{rw}}} \tag{37}$$

We can then use the motor's inbuilt Hall sensors to measure the reaction wheel's current angular velocity and, combined with the required change in angular velocity, determine the target velocity to produce the control torque. Once the target velocity is determined a simple PD controller is used to correct this angular velocity error by sending the appropriate PWM signals to the motor.



(a) D-TD3-R HIL path. (b) D-TD3-R HIL position error. (c) D-TD3-A HIL angular error.

(d) D-TD3-R HIL control forces. (e) D-TD3-A HIL control torques.

Fig. 18. HIL results for the TD3 control system.

7.2. The attitude dynamics testing platform

The design and assembly of the air bearing simulator revolves around two main parts; a stator, and a hemisphere. The stator should be able to accept compressed air, at a maximum pressure of 6 bar, and redirect it to the openings in its face creating a cushion of air for the hemisphere to rest on. There are two downsides issues with purchasing spherical air bearings; they are prohibitively expensive and they have long lead times, making them infeasible for some. Alternatively, there have been a few papers such as Jovanovic, Pearce, and Praks (2019), Molina, Hernández-Arias, Vera-Mendoza, Gonzalez, and Prado-Morales (2018), and Yanyachi, Mamani-Valencia, and Espinoza-García (2022) which show that it is possible to construct a hemispherical air bearing attitude testing platform at a significantly reduced cost over purchasing them. These in-house methods involve 3D printing most components and purchasing the hemisphere separately to ensure that it is smooth enough to function correctly.

The design of the stator is quite simple as its main function is to guide compressed air to openings in its face. The placement of the openings in the stator is essential to maintain a stable and steady stream of air. It was decided to follow the recommendation given by the authors of Jovanovic et al. (2019) and have a circular pattern of openings located within a channel around the surface of the stator. 3D models of the stator can be seen in Figs. 14 and 15

The stator will be 3D printed in two parts, with its bottom half being printed using an Onyx FFF printer and its top half printed using a Formlabs Fuse 1 SLS 3D printer to ensure a smoother and more precise print. The hemisphere will be purchased from a supplier and made of acrylic to ensure that it is smooth and remains lightweight. A hemisphere was deemed appropriate for testing as only small to medium angle slew manoeuvres would be tested but this could be easily amended in the future by replacing the hemisphere with a two-part sphere ensuring that the seam between both parts does not cause unwanted disturbances. The pneumatic components include various pneumatic fittings, hosing, and a pneumatic regulator to regulate the air pressure from the source and ensure a controlled airflow of 6 bar to the stator. In Fig. 16 the completed air-bearing platform containing the mock CubeSat is shown.

7.3. The rendezvous mission simulator

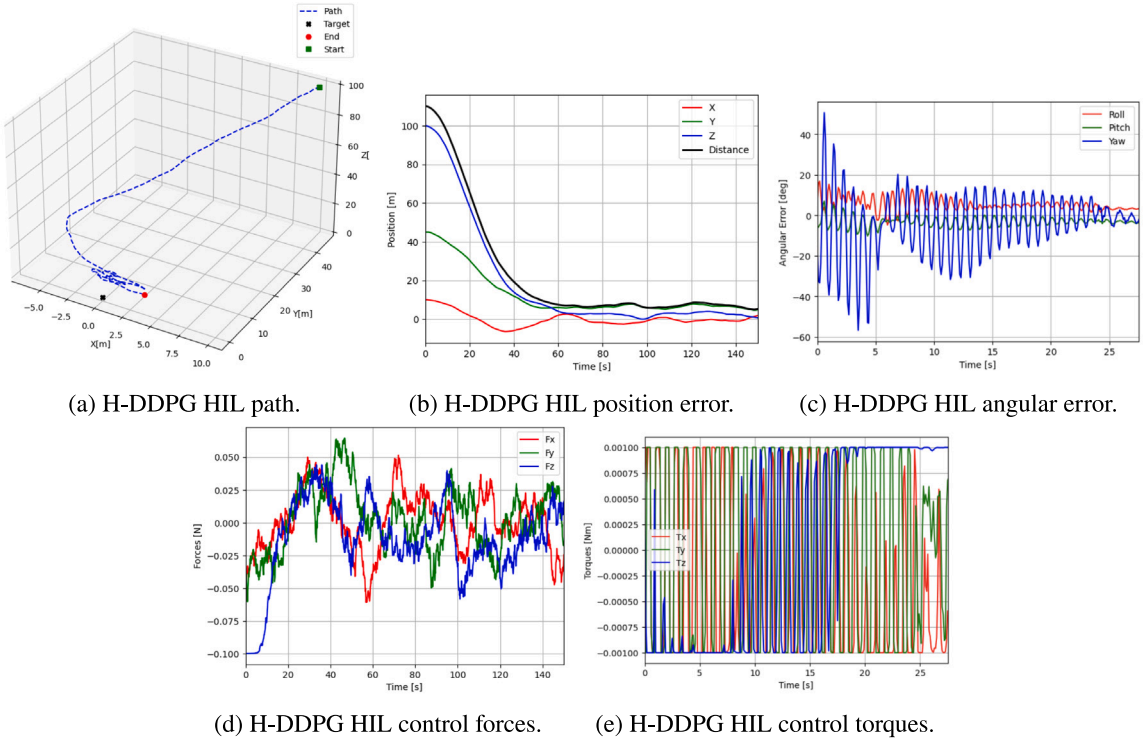
The rendezvous mission simulator is designed to emulate a CubeSat's operation in space, having a robotic arm acting as the thrustor for the mock CubeSat. The robotic arm is controlled by a central PC, where every 0.1 s it gathers the current position and velocity data of the arms end effector, it uses this state information as input to the H-DDPG-R agent hosted on the central PC. The agent outputs a force command, which is then translated into joint torques using the robot's Jacobian matrix to actuate the satellite. This setup allows for a scaled down simulation of spacecraft rendezvous operations, with the arm's movements mimicking those of a spacecraft thruster.

The robotic arm chosen for this platform is the Kinova Gen3 6DoF robotic arm. The testing platform includes a 3D-printed attachment for the arm that allows for the integration of the mock cubesat and the attitude testing platform onto the arm's end effector, facilitating simultaneous testing of both guidance and attitude control systems. Fig. 17 illustrates the setup of the rendezvous testing platform, demonstrating the robotic arm with the attitude dynamics testing platform and the mock cubesat fixed to its end effector.

7.4. HIL testing results

The results of the Hardware-in-the-Loop testing for both the D-TD3 and H-DDPG control systems are illustrated in Figs. 18 and 19, respectively. Both systems successfully achieved their attitude and guidance objectives in comparable timeframes despite the presence of disturbances. The impact of noise was most noticeable in the attitude dynamics platform, where vibrations from the reaction wheel motors and movements of the robotic arm's end effector caused the CubeSat's IMU to output noisy data.

In addition to the D-TD3 and H-DDPG controllers, we also conducted HIL testing using the same PD controller that was used in



(a) H-DDPG HIL path. (b) H-DDPG HIL position error. (c) H-DDPG HIL angular error.

(d) H-DDPG HIL control forces. (e) H-DDPG HIL control torques.

Fig. 19. HIL results for the H-DDPG control system.

the simulation results outlined in Section 6.2. Similarly to the testing in simulation the PD controller was unable to correct the CubeSat's attitude during hardware testing and performed comparably to the HDRL controller during the guidance task. These results suggest that HDRL controllers can be advantageous in real-world conditions, where classical controllers like the PD controller may struggle to provide the flexibility needed to manage the complexities of uncertain and dynamic environments.

In the case of the D-TD3 control system, significant angular errors were observed at the start of the test, with the system oscillating and exhibiting overshoots when attempting to correct these errors. By the end of the test, the angular errors were reduced to below 5° for the Pitch and Roll axes, and approximately 7.5° for the Yaw axis. During the guidance task, disturbances became more pronounced as the relative distance between the chaser and target decreased below 20 metres. Despite this, the system successfully brought the chaser to within 5 metres of the target in under 120 s. One notable issue was the presence of chattering in the commanded forces during the guidance task, a behaviour also observed during simulation.

Comparatively, the H-DDPG controller reduced angular errors to below 5° within 25 s, similar to the performance of the D-TD3 controller but with a lower final error across all axes. In the guidance task, the H-DDPG controller achieved a relative position of 5 metres from the target in under 90 s, slightly outperforming the D-TD3 system. Additionally, the H-DDPG controller exhibited less fluctuation in commanded forces and commanded lower forces overall, indicating a more stable response to disturbances.

Overall, both systems demonstrated effective control capabilities in the presence of disturbances, meeting their objectives within similar timeframes. However, the H-DDPG controller exhibited superior performance overall, achieving lower final angular errors, reduced chattering in control forces, and completing the guidance task more quickly. These results highlight the H-DDPG system's effectiveness in managing realistic disturbances and noise while fulfilling its guidance and control objectives.

8. Conclusion

In this paper, we introduced a novel Hierarchical Deep Reinforcement Learning controller for CubeSat guidance and attitude control, leveraging a Hierarchical Actor–Critic framework. The HDRL controller's performance was compared in simulation with a centralised and a decentralised TD3-based controller. To validate the HDRL controller's effectiveness, we conducted Hardware-in-the-Loop testing using a custom-built attitude and guidance testing platform. The results demonstrated that the HDRL controller is effective in managing both guidance and attitude control tasks, achieving precise positioning and attitude corrections while efficiently handling real-world disturbances such as sensor noise and actuator vibrations. This work contributes to advancing the state of the art in autonomous spacecraft technologies, providing a foundation for future research and applications in complex and dynamic space environments. As we continue to explore the potential of reinforcement learning in space applications, future research will extend the mission scenario to include the docking phase of satellite rendezvous and will aim to increase the controllers efficiency by decreasing actuator effort.

CRediT authorship contribution statement

Abdulla Tammam: Writing – review & editing, Writing – original draft, Visualisation, Validation, Software, Methodology, Investigation, Data curation, Conceptualisation. **Nabil Aouf:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

AlMahamid, F., & Grolinger, K. (2022). Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review. *Engineering Applications of Artificial Intelligence*, *115*, Article 105321. http://dx.doi.org/10.1016/j.engappai.2022.105321, URL https://www.sciencedirect.com/science/article/pii/S095219762200358X.

Aman, A., Arelhi, R., & Khan, N. (2019). Studying the effects of disturbance torques on a 2U CubeSat in low earth orbits. *Journal of Physics: Conference Series*, *1152*, Article 012024. http://dx.doi.org/10.1088/1742-6596/1152/1/012024.

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., et al. (2017). Hindsight experience replay. http://dx.doi.org/10.48550/ARXIV.1707.01495, URL https://arxiv.org/abs/1707.01495.

Azar, A. T., Koubaa, A., Ali Mohamed, N., Ibrahim, H. A., Ibrahim, Z. F., Kazim, M., et al. (2021). Drone deep reinforcement learning: A review. *Electronics*, *10*(9), http://dx.doi.org/10.3390/electronics10090999, URL https://www.mdpi.com/2079-9292/10/9/999.

Center, G. A. (2023). European proximity operations simulator (EPOS 2.0). URL https://www.dlr.de/en/research-and-transfer/research-infrastructure/european-proximity-operations-simulator-epos.

Cheng, C.-H., Shu, S.-L., & Cheng, P.-J. (2009). Attitude control of a satellite using fuzzy controllers. *Expert Systems with Applications*, *36*(3, Part 2), 6613–6620. http://dx.doi.org/10.1016/j.eswa.2008.08.053, URL https://www.sciencedirect.com/science/article/pii/S0957417408005988.

Cihang, R., & Jang, J.-S. (1994). *vol. 3*, *Fuzzy logic attitude control for Cassini spacecraft* (pp. 1532 – 1537). http://dx.doi.org/10.1109/FUZZY.1994.343922.

Cookson, J. (2019). *Experimental investigation of spacecraft rendezvous and docking by development of a 3 degree of freedom satellite simulator testbed*

Dankwa, S., & Zheng, W. (2019). Twin-delayed DDPG: A deep reinforcement learning technique to model a continuous movement of an intelligent robot agent. In *Proceedings of the 3rd international conference on vision, image and signal processing*. New York, NY, USA: Association for Computing Machinery, URL https://doi.org/10.1145/3387168.3387199.

Ding, Z., Yu, T., Huang, Y., Zhang, H., Mai, L., & Dong, H. (2020). RLzoo: A comprehensive and adaptive reinforcement learning library. arXiv preprint arXiv:2009.08644.

Faulhaber (2024). Faulhaber motors with integrated electronics. URL https://www.faulhaber.com/en/products/series/2610b-sc/.

Fear, A., & Lightsey, E. G. (2021). Implementation of small satellite autonomous rendezvous using model predictive control. http://dx.doi.org/10.2514/6.2022-0838. URL https://arc.aiaa.org/doi/abs/10.2514/6.2022-0838.

Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. http://dx.doi.org/10.48550/ARXIV.1802.09477, URL https://arxiv.org/abs/1802.09477.

Gavrilovich, I. (2016). *Development of a robotic system for cubeSat attitude determination and control system ground tests* (Ph.D. thesis).

Gui, H., & Vukovich, G. (2017). Adaptive fault-tolerant spacecraft attitude control using a novel integral terminal sliding mode. *International Journal of Robust and Nonlinear Control*, *27*(16), 3174–3196. http://dx.doi.org/10.1002/rnc.3733, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/rnc.3733, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/rnc.3733.

Gupta, R., Kalabić, U. V., Di Cairano, S., Bloch, A. M., & Kolmanovsky, I. V. (2015). Constrained spacecraft attitude control on SO(3) using fast nonlinear model predictive control. In *2015 American control conference* (pp. 2980–2986). http://dx.doi.org/10.1109/ACC.2015.7171188.

Hadi, R. (2015). *Development of dynamic simulation platform of reaction wheels controlled cubesat model* (Ph.D. thesis), http://dx.doi.org/10.13140/RG.2.1.4113.5841.

He, L., Aouf, N., Whidborne, J. F., & Song, B. (2020). Integrated moment-based LGMD and deep reinforcement learning for UAV obstacle avoidance. In *2020 IEEE international conference on robotics and automation* (pp. 7491–7497). http://dx.doi.org/10.1109/ICRA40945.2020.9197152.

Hegrenæs, Ø., Gravdahl, J. T., & Tøndel, P. (2005). Spacecraft attitude control using explicit model predictive control. *Automatica*, *41*(12), 2107–2114. http://dx.doi.org/10.1016/j.automatica.2005.06.015, URL https://www.sciencedirect.com/science/article/pii/S0005109805002657.

Hilton, W. F. (1986). Spacecraft attitude dynamics. P. C. Hughes. John Wiley. *Aeronautical Journal*, *90*(894), http://dx.doi.org/10.1017/S0001924000015578, 152–152.

Huang, X., & Duan, G. (2020). Fault-tolerant attitude tracking control of combined spacecraft with reaction wheels under prescribed performance. *ISA Transactions*, *98*, 161–172. http://dx.doi.org/10.1016/j.isatra.2019.08.041, URL https://www.sciencedirect.com/science/article/pii/S0019057819303878.

Izzo, D., Martens, M., & Pan, B. (2018). A survey on artificial intelligence trends in spacecraft guidance dynamics and control. CoRR abs/1812.02948. arXiv:1812.02948, URL http://arxiv.org/abs/1812.02948.

Jaekel, S., Lampariello, R., Rackl, W., De Stefano, M., Oumer, N., Giordano, A. M., et al. (2018). Design and operational elements of the robotic subsystem for the e.deorbit debris removal mission. *Frontiers in Robotics and AI*, *5*, http://dx.doi.org/10.3389/frobt.2018.00100, URL https://www.frontiersin.org/articles/10.3389/frobt.2018.00100.

Jovanovic, N., Pearce, J., & Praks, J. (2019). Design and testing of a low-cost, open source, 3-D printed air-bearing-based attitude simulator for CubeSat satellites.

Karr, C. L., & Freeman, L. (1997). Genetic-algorithm-based fuzzy control of spacecraft autonomous rendezvous. *Engineering Applications of Artificial Intelligence*, *10*(3), 293–300. http://dx.doi.org/10.1016/S0952-1976(97)00008-0, URL https://www.sciencedirect.com/science/article/pii/S0952197697000080.

Krishna, N. S., Gosavi, S., Singh, S., Saxena, N., Kailaje, A., Datla, V., et al. (2018). Design and implementation of a reaction wheel system for CubeSats. In *2018 IEEE aerospace conference* (pp. 1–7). http://dx.doi.org/10.1109/AERO.2018.8396584.

Levy, A., Konidaris, G., Platt, R., & Saenko, K. (2017). Learning multi-level hierarchies with hindsight. http://dx.doi.org/10.48550/ARXIV.1712.00948, URL https://arxiv.org/abs/1712.00948.

Levy, A., Platt, R., Jr., & Saenko, K. (2017). Hierarchical actor-critic. CoRR abs/1712.00948. arXiv:1712.00948, URL http://arxiv.org/abs/1712.00948.

Li, Q., Yuan, J., Zhang, B., & Gao, C. (2017). Model predictive control for autonomous rendezvous and docking with a tumbling target. *Aerospace Science and Technology*, *69*, 700–711. http://dx.doi.org/10.1016/j.ast.2017.07.022, URL https://www.sciencedirect.com/science/article/pii/S1270963817301293.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., et al. (2015). Continuous control with deep reinforcement learning. http://dx.doi.org/10.48550/ARXIV.1509.02971, URL https://arxiv.org/abs/1509.02971.

Long, F. W. (2014). Design and testing of a nanosatellite simulator reaction wheel attitude control system. URL https://api.semanticscholar.org/CorpusID:106528793.

Luo, Y., Zhang, J., & Tang, G. (2014). Survey of orbital dynamics and control of space rendezvous. *Chinese Journal of Aeronautics*, *27*(1), 1–11. http://dx.doi.org/10.1016/j.cja.2013.07.042, URL https://www.sciencedirect.com/science/article/pii/S1000936113001787.

Luzi, A.-R., Peaucelle, D., Biannic, J., Pittet, C., & Mignot, J. (2014). Structured adaptive attitude control of a satellite. *International Journal of Adaptive Control and Signal Processing*, *28*(7–8), 664–685. http://dx.doi.org/10.1002/acs.2406, URL https://hal.science/hal-01760872.

Markley, L., & Crassidis, J. (2014). Fundamentals of spacecraft attitude determination and control. ISBN: 978-1-4939-0802-8, http://dx.doi.org/10.1007/978-1-4939-0802-8.

Mazal, L., Pérez, D., Bevilacqua, R., & Curti, F. (2016). Spacecraft rendezvous by differential drag under uncertainties. *Journal of Guidance, Control, and Dynamics*, *39*(8), 1721–1733. http://dx.doi.org/10.2514/1.G001785, arXiv:https://doi.org/10.2514/1.G001785.

Molina, J., Hernández-Arias, H., Vera-Mendoza, D., Gonzalez, J., & Prado-Morales, J. (2018). Frictionless spacecraft simulator with unrestricted three-axis movement for nanosats. *International Journal of Scientific and Technology Research*, *7*, 84–95.

Navabi, M., & Radaei, M. (2013). Attitude adaptive control of space systems. In *2013 6th international conference on recent advances in space technologies* (pp. 973–977). http://dx.doi.org/10.1109/RAST.2013.6581356.

Newton, A. (2021). *Design, development, and experimental validation of a nanosatellite attitude control simulator* (Ph.D. thesis).

Oland, E., & Schlanbusch, R. (2009). Reaction wheel design for CubeSats. In *2009 4th international conference on recent advances in space technologies* (pp. 778–783). http://dx.doi.org/10.1109/RAST.2009.5158296.

Ortega, G. (1995). Fuzzy logic techniques for rendezvous and docking of two geostationary satellites. *Telematics and Informatics*, *12*(3), 213–227. http://dx.doi.org/10.1016/0736-5853(95)00013-5, URL https://www.sciencedirect.com/science/article/pii/0736585395000135, Advanced Space Technologies For Systems Autonomy.

Pearson, D. J. (1989). The glideslope approach.

Pham, H. X., La, H. M., Feil-Seifer, D., & Nguyen, L. V. (2018). Autonomous UAV navigation using reinforcement learning. CoRR abs/1801.05086. arXiv:1801.05086, URL http://arxiv.org/abs/1801.05086.

Regan, F. J., & Anandakrishnan, S. M. (1993). *Dynamics of atmospheric re-entry*. Washington DC: American Institute of Aeronautics and Astronautics.

Roberson, R. E. (1964). Generalized gravity-gradient torques. In S. F. Singer (Ed.), *Applied mathematics and mechanics: vol. 7*, *Torques and attitude sensing in earth satellites* (pp. 73–82). Elsevier, http://dx.doi.org/10.1016/B978-0-12-395776-4.50009-4, URL https://www.sciencedirect.com/science/article/pii/B9780123957764500094.

Santaguida, L., & Zhu, Z. H. (2023). Development of air-bearing microgravity testbed for autonomous spacecraft rendezvous and robotic capture control of a free-floating target. *Acta Astronautica*, *203*, 319–328. http://dx.doi.org/10.1016/j.actaastro.2022.11.056, URL https://www.sciencedirect.com/science/article/pii/S0094576522006725.

Schaul, T., Horgan, D., Gregor, K., & Silver, D. (2015). Universal value function approximators. In F. Bach, & D. Blei (Eds.), *Proceedings of machine learning research*: *vol. 37*, *Proceedings of the 32nd international conference on machine learning* (pp. 1312–1320). Lille, France: PMLR, URL https://proceedings.mlr.press/v37/schaul15.html.

Shirobokov, M., Trofimov, S., & Ovchinnikov, M. (2021). Survey of machine learning techniques in spacecraft control design. *Acta Astronautica*, *186*, 87–97. http://dx.doi.org/10.1016/j.actaastro.2021.05.018, URL https://www.sciencedirect.com/science/article/pii/S0094576521002514.

Singla, A., Padakandla, S., & Bhatnagar, S. (2021). Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge. *IEEE Transactions on Intelligent Transportation Systems*, *22*(1), 107–118. http://dx.doi.org/10.1109/TITS.2019.2954952.

Song, J., Rondao, D., & Aouf, N. (2022). Deep learning-based spacecraft relative navigation methods: A survey. *Acta Astronautica*, *191*, 22–40. http://dx.doi.org/10.1016/j.actaastro.2021.10.025.

Systems, E. (2023). ESP32-devkitc V2 getting started guide. URL https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc-v2.html.

Tipaldi, M., & Glielmo, L. (2018). A survey on model-based mission planning and execution for autonomous spacecraft. *IEEE Systems Journal*, *12*(4), 3893–3905. http://dx.doi.org/10.1109/JSYST.2017.2720682.

Tipaldi, M., Iervolino, R., & Massenio, P. R. (2022). Reinforcement learning in spacecraft control applications: Advances, prospects, and challenges. *Annual Reviews in Control*, *54*, 1–23. http://dx.doi.org/10.1016/j.arcontrol.2022.07.004, URL https://www.sciencedirect.com/science/article/pii/S136757882200089X.

Tiwari, P. M., Janardhanan, S., & un Nabi, M. (2015). Rigid spacecraft attitude control using adaptive integral second order sliding mode. *Aerospace Science and Technology*, *42*, 50–57. http://dx.doi.org/10.1016/j.ast.2014.11.017, URL https://www.sciencedirect.com/science/article/pii/S1270963815000024.

Wang, S.-C. (2003). Artificial neural network. In *Interdisciplinary computing in java programming* (pp. 81–100). Boston, MA: Springer US, http://dx.doi.org/10.1007/978-1-4615-0377-4_5.

Wertz, J. R. (1990). Attitude prediction. In *Spacecraft attitude determination and control* (pp. 558–559). Boston: Kluwer Academic Publishers.

White, C. C., Gass, S. I., & Harris, C. M. (2001). Markov decision processes Markov decision processes. In *Encyclopedia of operations research and management science* (pp. 484–486). New York, NY: Springer US, http://dx.doi.org/10.1007/1-4020-0611-X_580.

Yanyachi, P. R., Mamani-Valencia, H., & Espinoza-García, B. (2022). Low-cost test system for 1U CubeSat attitude control with reaction wheels. In *2022 IEEE biennial congress of Argentina* (pp. 1–8). http://dx.doi.org/10.1109/ARGENCON55245.2022.9940099.

Zheng, M., Wu, Y., & Li, C. (2021). Reinforcement learning strategy for spacecraft attitude hyperagile tracking control with uncertainties. *Aerospace Science and Technology*, *119*, Article 107126. http://dx.doi.org/10.1016/j.ast.2021.107126, URL https://www.sciencedirect.com/science/article/pii/S1270963821006362.



Abdulla Tammam is a Ph.D. candidate at City, University of London, with a focus on the Guidance, Navigation, and Control of nanosatellites. Abdulla holds an IET-accredited bachelor's degree in Electrical and Electronic Engineering and is a member of the Robotics, Autonomy, and Machine Intelligence (RAMI) research group. His research interests span machine autonomy, reinforcement learning, and robust control. He has also contributed as a Space Policy Project Assistant in association with the European Space Agency (ESA), focusing on space liability and the adoption of AI technologies. Additionally, he is actively involved in a NATO-supported autonomous drone flying competition, participating in international challenges and contributing to event organisation.



Prof. Nabil Aouf received his Ph.D. from McGill University in 2002 at the Electrical and Computer Engineering Department. Currently, he is Professor of Autonomous Systems and Machine Intelligence at City University of London. He is the Director of the Systems, Autonomy and Control (SAC) Centre and the co-Director of the London Space Institute (LSI) at City University of London. He also leads the Robotics, Autonomy and Machine Intelligence (RAMI) group and works very closely with industries that have a strong heritage in autonomous systems and space research. He has authored over 180 high calibre publications in his domains of interest. His research interests are aerospace and defence systems, information fusion and vision systems, guidance and navigation, control, and autonomy of systems. He is an Associate Editor of 4 journals including IEEE Transactions of Intelligent Vehicles.